# Smart Tank Monitoring System

Relazione tecnica – Assignment 03

Embedded Systems and Internet of Things
Anno accademico 2025/2026

Docente: Alessandro Ricci


| **Autori** | **Matricole** |
|---|---|
| Lucas Prati | 0001117339 |
| Samuele D'Ambrosio | 0001125837 |


6 novembre 2026

# Indice


**1 Introduzione e obiettivi** **2**
- 1.1 Contesto e obiettivi del progetto . . . 2
- 1.2 Comportamento e requisiti principali . . . 2

**2 Architettura generale del sistema** **3**
- 2.1 Componenti e confini di responsabilità . . . 3
- 2.2 Comunicazione e propagazione dello stato . . . 3
- 2.3 Stato globale e disponibilità . . . 3
- 2.4 Scelte architetturali . . . 4

**3 Tank Monitoring Subsystem (TMS)** **5**
- 3.1 Architettura software . . . 5
- 3.2 Circuito e componenti hardware . . . 5
- 3.3 Task del firmware . . . 7
- 3.4 Scelte progettuali e gestione degli errori . . . 10
- 3.5 Test, criticità e taratura . . . 11

**4 Control Unit Subsystem (CUS)** **12**
- 4.1 Architettura software . . . 12
- 4.2 Gestione dello stato e politica automatica . . . 13
- 4.3 Comunicazione e sincronizzazione . . . 15
- 4.4 Validazione, test e limitazioni . . . 15

**5 Water Channel Subsystem (WCS)** **16**
- 5.1 Architettura e hardware . . . 16
- 5.2 Scheduler, task e macchine a stati . . . 17
- 5.3 Interfaccia operatore e controllo della valvola . . . 18
- 5.4 Validazione, test e limitazioni . . . 18

**6 Dashboard Subsystem (DBS)** **19**
- 6.1 Architettura dell'interfaccia web . . . 19
- 6.2 Comunicazione HTTP e gestione dello stato . . . 19
- 6.3 Interazione dell'operatore . . . 19
- 6.4 Validazione, test e limitazioni . . . 20

**7 Integrazione e comunicazione** **21**
- 7.1 Interazione tra sottosistemi e contratti . . . 21
- 7.2 Scenari operativi e gestione delle interruzioni . . . 22
- 7.3 Validazione end-to-end e copertura dei requisiti . . . 23

**8 Conclusioni** **24**
- 8.1 Risultati e limitazioni . . . 24
- 8.2 Possibili sviluppi futuri . . . 24

# 1 Introduzione e obiettivi

## 1.1 Contesto e obiettivi del progetto

Il progetto realizza un prototipo distribuito per monitorare il livello dell'acqua piovana in una tanica e controllare l'apertura di un canale di scarico. Il sistema prevede l'acquisizione periodica delle misure, il controllo automatico dell'apertura e la possibilità di intervento da parte dell'operatore, sia localmente sia attraverso un'interfaccia web.

La consegna dell'Assignment 03 definisce i requisiti funzionali, i sottosistemi e le modalità di comunicazione previste [1]. Il sistema è composto da quattro sottosistemi: il Tank Monitoring Subsystem (TMS), responsabile dell'acquisizione delle misure; il Control Unit Subsystem (CUS), che coordina il funzionamento del sistema; il Water Channel Subsystem (WCS), che gestisce l'attuatore e l'interfaccia locale; e il Dashboard Subsystem (DBS), che permette il monitoraggio e il controllo remoto.

## 1.2 Comportamento e requisiti principali

Il sistema si avvia in modalità `AUTOMATIC`, nella quale l'apertura del canale viene regolata in base al livello dell'acqua e alle soglie temporali previste. In modalità `MANUAL`, l'operatore può impostare direttamente l'apertura desiderata. È inoltre prevista la gestione dell'indisponibilità delle misure e delle comunicazioni.

Il prototipo utilizza un ESP32 per il TMS, un Arduino UNO per il WCS, un backend Java eseguito su PC per il CUS e un'interfaccia web per il DBS. Le comunicazioni con il CUS avvengono rispettivamente tramite MQTT, connessione seriale e HTTP.

# 2 Architettura generale del sistema

## 2.1 Componenti e confini di responsabilità

Il sistema adotta un'architettura distribuita con controllo centralizzato nel CUS, responsabile della gestione dello stato globale, delle modalità operative e della determinazione dell'apertura della valvola. Le responsabilità dei diversi sottosistemi sono illustrate nella Figura 1.

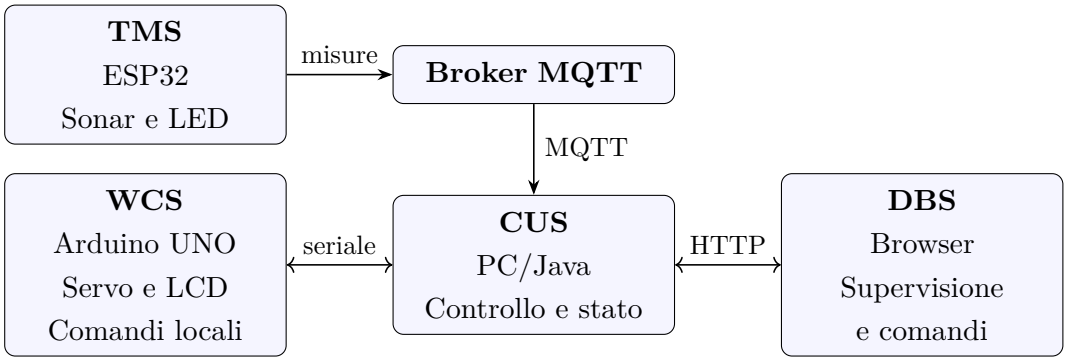



**Figura 1:** Architettura del sistema e collegamenti tra i sottosistemi.

Il TMS acquisisce il livello dell'acqua tramite un sensore sonar e trasmette le misure al CUS. Quest'ultimo elabora i dati ricevuti, applica la politica di controllo e rende disponibile lo stato del sistema. Il WCS gestisce l'attuatore e l'interfaccia locale, consentendo all'operatore di richiedere il cambio di modalità tramite pulsante e la regolazione dell'apertura tramite potenziometro. Il DBS permette il monitoraggio e il controllo remoto attraverso un'interfaccia web.

## 2.2 Comunicazione e propagazione dello stato

La comunicazione tra TMS e CUS avviene tramite MQTT, utilizzando un broker per lo scambio delle misure. Il WCS comunica con il CUS attraverso una connessione seriale, inviando le richieste dell'operatore e ricevendo gli aggiornamenti di stato necessari al controllo dell'attuatore e del display.

Il DBS comunica con il CUS tramite HTTP, recuperando periodicamente lo stato del sistema e lo storico delle misure. Attraverso la stessa interfaccia può inviare richieste di cambio modalità e regolazione dell'apertura.

I dettagli dei protocolli e dei messaggi sono descritti nella Sezione 7.1.

## 2.3 Stato globale e disponibilità

Il CUS gestisce le modalità operative `AUTOMATIC` e `MANUAL` e monitora la disponibilità delle misure provenienti dal TMS. In assenza di misure valide per un intervallo di tempo prestabilito, il sistema passa allo stato `UNCONNECTED`.

Il DBS distingue questa condizione dalla propria indisponibilità di comunicazione con il CUS, rappresentata come `NOT AVAILABLE`. Il backend può quindi essere raggiungibile anche quando le misure del TMS non sono disponibili.

## 2.4 Scelte architetturali

Le richieste degli operatori locali e remoti vengono valutate dal CUS in base allo stato corrente, senza una priorità prestabilita tra le due interfacce.

Nel backend, la logica di controllo è separata dai servizi di comunicazione. L'accesso allo stato condiviso è sincronizzato per gestire le operazioni concorrenti, mantenendo le operazioni di comunicazione al di fuori delle sezioni critiche.

# 3 Tank Monitoring Subsystem (TMS)

Il Tank Monitoring Subsystem (TMS) acquisisce il livello di riempimento della tanica e comunica le misure al CUS tramite MQTT.

## 3.1 Architettura software

Il firmware adotta un'architettura task-based basata su FreeRTOS, con due task eseguiti su core distinti. Il `WaterLevelTask`, assegnato al core 0, effettua le misurazioni ed elabora il livello dell'acqua. Il `CommunicationTask`, assegnato al core 1, gestisce la connessione Wi-Fi e MQTT e la pubblicazione dei risultati. Entrambi i task sono configurati con priorità 1.

La Tabella 1 riassume le responsabilità dei due task.

| Task | Responsabilità principale |
|---|---|
| `WaterLevelTask` | Acquisizione della distanza, verifica della validità e calcolo del livello di riempimento. |
| `CommunicationTask` | Gestione delle connessioni Wi-Fi e MQTT, pubblicazione delle misure e aggiornamento dei LED di stato. |

**Tabella 1:** Task principali del firmware TMS.

Lo scambio dei dati avviene tramite una coda FreeRTOS con capacità di un elemento di tipo `WaterLevelData`. Il task di misurazione inserisce il risultato più recente, sostituendo un'eventuale misura non ancora prelevata; il task di comunicazione lo legge quando può procedere alla pubblicazione. Questa separazione permette di continuare l'acquisizione anche durante l'assenza della connessione, senza accumulare una sequenza di misure precedenti da trasmettere.

## 3.2 Circuito e componenti hardware

Il prototipo TMS utilizza i seguenti componenti principali:

- una scheda ESP32, per l'esecuzione del firmware e la connessione Wi-Fi;
- un sensore a ultrasuoni HC-SR04, per misurare la distanza dalla superficie dell'acqua;
- un LED verde e un LED rosso, per segnalare lo stato della comunicazione;
- breadboard, cavetti e resistori per la realizzazione dei collegamenti.

La configurazione dei pin è centralizzata nel file `config.h`, così da mantenere separati i collegamenti hardware dalla logica dei task. La Figura 2 mostra lo schema del cablaggio.

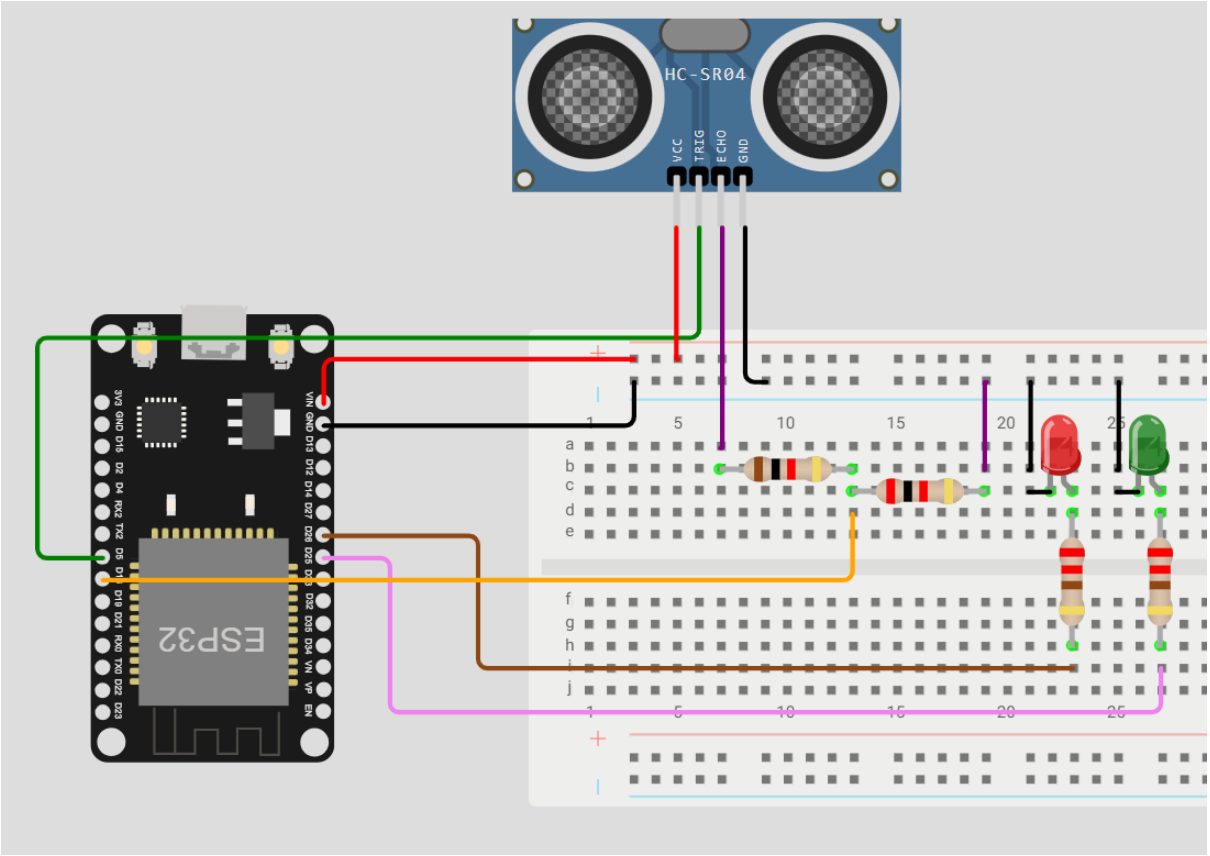



**Figura 2:** Cablaggio del TMS: ESP32, sonar HC-SR04 e LED di stato.

La Tabella 2 riassume i collegamenti dei segnali.

| GPIO | Collegamento |
|---|---|
| 5 | Segnale TRIG del sonar HC-SR04. |
| 18 | Segnale ECHO del sonar HC-SR04, attraverso il partitore resistivo. |
| 25 | LED verde di stato della comunicazione. |
| 26 | LED rosso di stato della comunicazione. |

**Tabella 2:** Assegnazione dei GPIO del TMS.

Il sonar è alimentato dalla linea a 5 V, con massa comune alla scheda ESP32. Sul segnale ECHO è presente un partitore formato da una resistenza da 1 kΩ tra l'uscita del sensore e il nodo collegato a GPIO18, e una resistenza da 2 kΩ tra tale nodo e massa. Il partitore riduce il livello alto del segnale da circa 5 V a circa 3,3 V, rendendolo compatibile con l'ingresso della scheda.

Ciascun LED è collegato al rispettivo GPIO attraverso una resistenza in serie da 220 Ω, che ne limita la corrente; i catodi sono collegati a massa.

## 3.3 Task del firmware

### 3.3.1 CommunicationTask

Il `CommunicationTask` gestisce la connessione alla rete Wi-Fi e al broker MQTT e la pubblicazione delle misure acquisite dal TMS. La logica di controllo è realizzata mediante una macchina a stati finiti, che distingue le fasi di connessione dalla condizione di comunicazione operativa.

Gli stati principali sono:

- `WIFI_CONNECTING`: attesa della connessione alla rete Wi-Fi configurata;
- `MQTT_CONNECTING`: tentativi di connessione al broker MQTT, rispettando l'intervallo di ritentativo configurato;
- `WAITING_FIRST_PUBLISH`: connessione MQTT stabilita, in attesa della prima pubblicazione riuscita;
- `COMMUNICATION_OK`: almeno una pubblicazione è riuscita dopo la connessione MQTT corrente; il task continua a gestire la connessione e a pubblicare le nuove misure disponibili.

La Figura 3 mostra la macchina a stati del task di comunicazione.

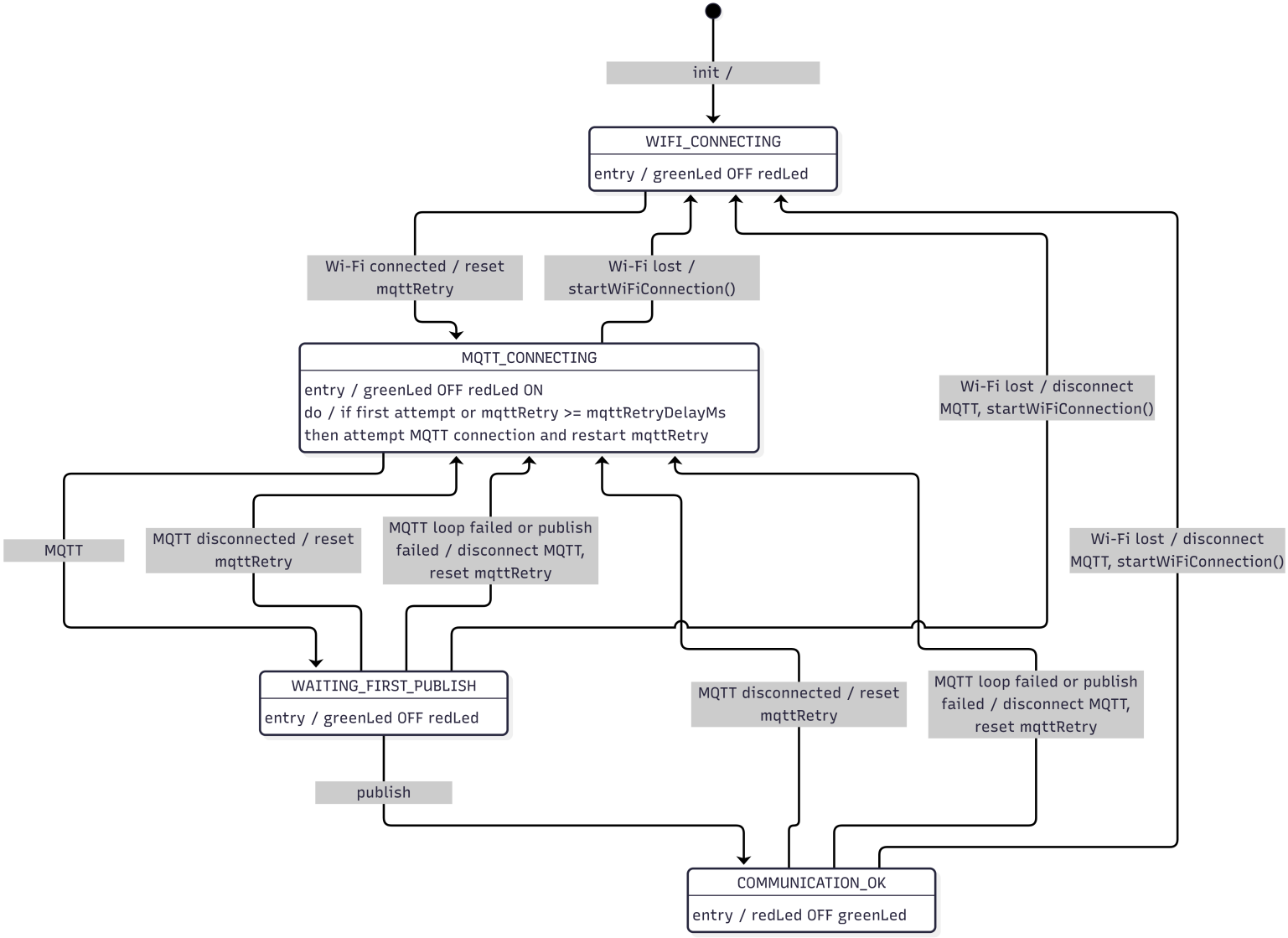



**Figura 3:** FSM del CommunicationTask.

Negli stati `WAITING_FIRST_PUBLISH` e `COMMUNICATION_OK`, il task esegue le stesse operazioni di gestione della connessione e pubblicazione. Richiama regolarmente `mqttClient.loop()` per mantenere la comunicazione MQTT e, quando trova una misura nella coda, la preleva e ne tenta la pubblicazione sul topic configurato. Il primo esito positivo determina il passaggio a `COMMUNICATION_OK`; le pubblicazioni successive proseguono nello stesso stato.

Il LED rosso rimane acceso durante la connessione Wi-Fi, la connessione MQTT e l'attesa della prima pubblicazione riuscita. In `COMMUNICATION_OK` viene spento il rosso e acceso il verde. Quando viene rilevata una perdita della connessione, il passaggio allo stato di riconnessione spegne il LED verde e riaccende il LED rosso. Il verde viene riacceso soltanto dopo una nuova pubblicazione riuscita. I LED rappresentano quindi lo stato della comunicazione, indipendentemente dalla validità della misura: anche l'esito di una lettura senza eco può essere pubblicato correttamente.

La perdita della connessione Wi-Fi determina il ritorno a `WIFI_CONNECTING`. Se il Wi-Fi è disponibile ma la connessione MQTT risulta interrotta, oppure fallisce la gestione del client o una pubblicazione, il task passa a `MQTT_CONNECTING`. Il ritorno a questo stato abilita un nuovo tentativo immediato; in caso di ulteriori fallimenti, i tentativi rispettano l'intervallo di ritentativo configurato.

Le misure vengono pubblicate in formato JSON sul topic definito dalla costante `MQTT_DATA_TOPIC` nel file `config.h`. Il CUS riceve i messaggi sottoscrivendo lo stesso topic.

Per una misura valida, il messaggio contiene lo stato `OK`, il livello dell'acqua espresso in metri nel campo `levelM` e la percentuale di riempimento nel campo `percentage`. Ad esempio, considerando l'altezza utile del prototipo di 0,30 m, un livello di 0,15 m corrisponde a un riempimento del 50% e produce:

```
{"status":"OK","levelM":0.150,"percentage":50.0}
```

Per una lettura non valida, il messaggio contiene soltanto l'indicazione di stato:

```
{"status":"NO_ECHO"}
```

In questo caso, i campi numerici del livello e della percentuale sono assenti. Il CUS può quindi distinguere una lettura non valida da una misura valida con riempimento pari allo 0%. La segnalazione viene pubblicata sullo stesso topic utilizzato per le misure valide.

### 3.3.2 WaterLevelTask

Il `WaterLevelTask` acquisisce la distanza dalla superficie dell'acqua attraverso il sonar e calcola il livello di riempimento della tanica. Il periodo di acquisizione è definito nel file `config.h` e impostato a 2000 ms, corrispondenti a una frequenza di campionamento di 0,5 Hz.

Nel modello adottato, il sensore è collocato in corrispondenza del piano che rappresenta il riempimento al 100%. Indicando con $H$ la distanza tra il sensore e il fondo della tanica e con $d$ la distanza misurata dalla superficie dell'acqua, il livello viene calcolato come:

$$h = H - d$$

La percentuale di riempimento è quindi ottenuta mediante:

$$p = \frac{h}{H} \cdot 100$$

La misura è considerata valida soltanto se la distanza rilevata è compresa tra zero e $H$. Per le letture ammesse, il livello calcolato appartiene già all'intervallo compreso tra zero e l'altezza della tanica, mentre la percentuale risulta compresa tra 0% e 100%.

Se la distanza è negativa, indicando l'assenza di eco, oppure supera l'altezza configurata della tanica, il task imposta il campo `measurementValid` a falso e non esegue il calcolo del livello. La lettura non valida rimane quindi distinta dalla condizione di tanica vuota, che corrisponde invece a una misura valida con distanza pari a $H$.

A ogni acquisizione, il task prepara una struttura `WaterLevelData`, contenente la validità della misura, la distanza rilevata, il livello e la percentuale di riempimento. La struttura viene inserita nella coda condivisa mediante `xQueueOverwrite()`, sostituendo l'eventuale risultato non ancora prelevato dal task di comunicazione. Anche le letture non valide vengono inserite nella coda: il flag di validità permette al task di comunicazione di distinguerle dalle misure utilizzabili.

La periodicità è gestita mediante `vTaskDelayUntil()`, che riferisce i risvegli del task a scadenze periodiche, anziché aggiungere un intervallo fisso al termine di ogni acquisizione.

## 3.4 Scelte progettuali e gestione degli errori

Le principali scelte progettuali riguardano la continuità dell'acquisizione, la gestione delle misure e la configurazione del prototipo:

- **Separazione tra acquisizione e comunicazione.** Le due attività sono affidate a task distinti, così che l'attesa della connessione Wi-Fi o MQTT non interrompa il campionamento del livello dell'acqua.
- **Priorità alla misura più recente.** La coda condivisa contiene un solo elemento e viene aggiornata mediante sovrascrittura. Questa scelta privilegia la disponibilità dello stato corrente della tanica, evitando l'accumulo di misure da trasmettere dopo un'interruzione della connessione.
- **Distinzione tra lettura non valida e tanica vuota.** La validità viene rappresentata esplicitamente nella struttura della misura. Una lettura non utilizzabile viene comunicata mediante lo stato `NO_ECHO`, mentre una tanica vuota corrisponde a una misura valida con livello pari a zero.
- **Configurazione centralizzata.** I pin, i parametri di rete, i periodi dei task e l'altezza utile della tanica sono raccolti nel file `config.h`. Il valore dell'altezza deve corrispondere alla distanza effettiva tra il sensore e il fondo, coerentemente con il modello utilizzato per il calcolo del livello.

Le interruzioni della connessione vengono gestite dalla macchina a stati del `CommunicationTask`, che effettua nuovi tentativi di connessione senza richiedere il

riavvio della scheda. Il LED rosso segnala la condizione di comunicazione non operativa durante il recupero.

Gli errori di inizializzazione vengono invece rilevati controllando il risultato della creazione della coda e dei task FreeRTOS. In caso di fallimento, viene stampato un messaggio diagnostico sulla porta seriale e viene sospeso il task Arduino che esegue `setup()`, mediante `vTaskSuspend(nullptr)`. La sospensione impedisce di proseguire l'inizializzazione con risorse mancanti e riguarda soltanto il task chiamante: gli eventuali task già creati rimangono attivi.

## 3.5 Test, criticità e taratura

**Allestimento e verifica delle misure.** Le prove del TMS sono state effettuate utilizzando una scatolina in polimero di forma geometrica regolare come bersaglio del sonar. Variando la distanza tra il sensore e l'oggetto sono stati simulati diversi livelli di riempimento della tanica.

L'altezza utile è stata configurata a 0,30 m, corrispondenti alla lunghezza del righello utilizzato come riferimento. La distanza rilevata dal sonar è stata confrontata con quella misurata mediante il righello, posizionando la scatolina a distanze differenti dal sensore. Le letture sono risultate approssimativamente corrispondenti alle distanze di riferimento, fornendo una verifica qualitativa dell'acquisizione nella configurazione utilizzata.

È stata verificata anche la coerenza tra il livello espresso in metri e la percentuale di riempimento, rispetto all'altezza configurata. Con un'altezza utile di 0,30 m, un livello di 0,15 m corrisponde a un riempimento del 50%.

**Gestione delle letture non valide.** Durante le prove preliminari con la mano come bersaglio sono state osservate letture non valide, segnalate come `NO_ECHO`. È stato quindi utilizzato un oggetto di forma regolare per proseguire le verifiche.

È stata controllata anche la gestione delle distanze superiori all'altezza configurata, collocando il bersaglio oltre i 30 cm. La lettura è stata correttamente classificata come non valida e comunicata mediante lo stato `NO_ECHO`. Per le letture non valide è stata verificata la gestione sul CUS, che ha riconosciuto la segnalazione e restituito `null` per la misura non disponibile. Il comportamento osservato distingue quindi una misura non utilizzabile da una misura valida con livello pari a zero.

**Avvio e recupero della comunicazione.** È stata verificata la segnalazione luminosa durante l'avvio del TMS e dopo un riavvio manuale mediante il pulsante RESET della scheda. In entrambi i casi è stata osservata l'accensione iniziale del LED rosso, seguita dal passaggio al verde al ripristino della comunicazione. Questa sequenza è coerente con la macchina a stati, che abilita il verde dopo la prima pubblicazione MQTT riuscita.

È stata inoltre verificata la gestione della perdita della connessione Wi-Fi durante il funzionamento, interrompendo temporaneamente la rete e mantenendo la scheda accesa. Al ripristino della rete, il TMS ha ristabilito automaticamente la comunicazione, tornando alla condizione operativa segnalata dal LED verde. Il recupero è avvenuto senza richiedere un riavvio manuale della scheda.

# 4 Control Unit Subsystem (CUS)

Il CUS costituisce il nodo centrale del sistema: riceve le misure dal TMS, gestisce le richieste degli operatori e determina l'apertura da comandare alla valvola.

## 4.1 Architettura software

Il CUS è sviluppato in Java 17 e utilizza Maven per la gestione del progetto. Il backend si basa su Vert.x per i servizi HTTP e i timer, Eclipse Paho per MQTT e jSerialComm per la comunicazione seriale.

La logica di controllo è concentrata in `SystemController`, che gestisce lo stato condiviso e coordina `AutomaticPolicy` e `MeasurementHistory`, responsabili rispettivamente della politica automatica e dello storico delle misure. I servizi MQTT, HTTP e seriale gestiscono le comunicazioni con gli altri sottosistemi, come illustrato nella Figura 4.

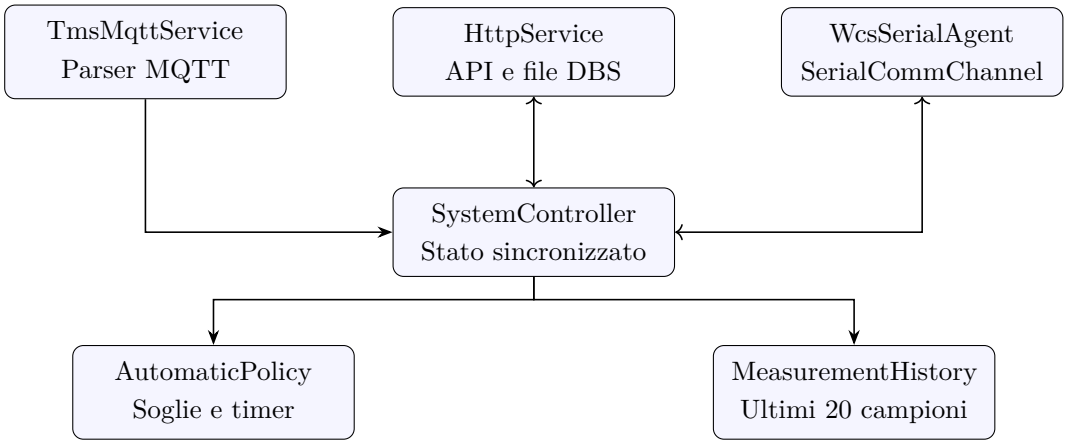



**Figura 4:** Organizzazione software del CUS e relazioni tra i componenti principali.

La configurazione viene letta dal file `cus.properties`, situato nella directory di avvio dell'applicazione. In assenza del file vengono utilizzati i valori predefiniti. Il percorso della directory DBS è relativo alla directory di esecuzione.

I principali parametri del CUS sono riportati nella Tabella 3. Le soglie, gli intervalli temporali T1 e T2 e la porta HTTP sono configurabili; il tick periodico e la capacità dello storico sono invece definiti nel codice.

| Parametro | Valore | Descrizione |
|---|---|---|
| $L_1, L_2$ | $40\%, 70\%$ | Soglie del livello dell'acqua. |
| $T_1$ | 5 s | Intervallo di attesa nella fascia intermedia. |
| $T_2$ | 10 s | Tempo massimo senza misure valide. |
| Tick | 100 ms | Intervallo di controllo periodico. |
| $N$ | 20 | Capacità dello storico delle misure. |
| HTTP | 8080 | Porta del servizio HTTP. |

**Tabella 3:** Parametri principali del CUS e relativi valori predefiniti o definiti nel codice.

## 4.2 Gestione dello stato e politica automatica

All'avvio il sistema si trova in modalità `AUTOMATIC`, con apertura della valvola allo 0% e livello dell'acqua non ancora disponibile. Il conteggio di T2 inizia all'avvio, in attesa della prima misura valida.

### 4.2.1 Politica a stati

La politica automatica è organizzata attraverso una FSM con cinque stati interni: `UNKNOWN`, stato iniziale o successivo a un reset; `LOW`, associato al livello basso; `MID_WAITING`, che rappresenta l'attesa T1; `MID_ACTIVE`, con apertura al 50%; e `HIGH`, con apertura al 100%.

Le condizioni applicate alle misure valide sono riassunte nella Tabella 4.

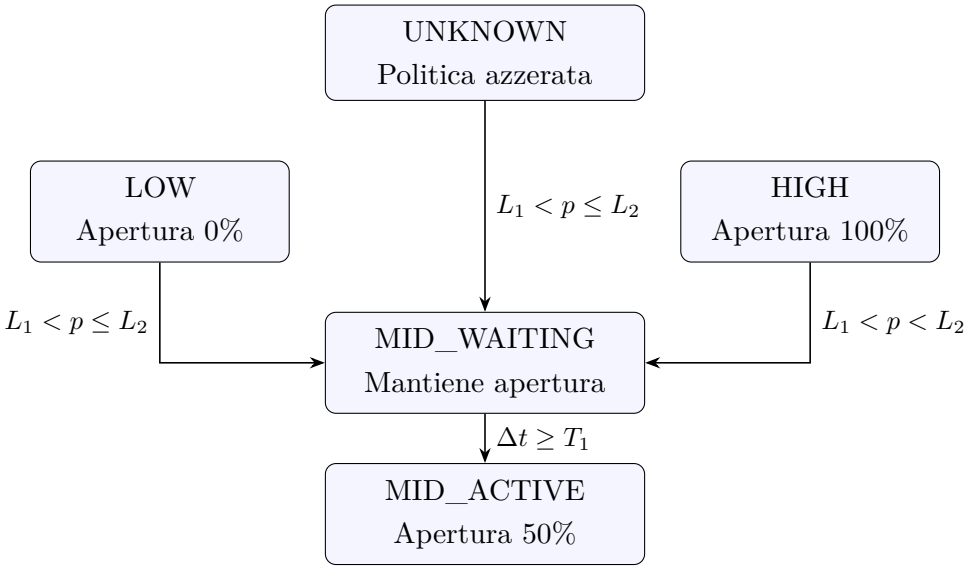
| Livello | Comportamento |
|---|---|
| $p < L_1$ | Passa a `LOW` e imposta l'apertura allo 0%. |
| $p = L_1$ | Mantiene il 50% se si trova già in `MID_ACTIVE`; altrimenti passa a `LOW` e chiude la valvola. |
| $L_1 < p < L_2$ | Mantiene il 50% se già in `MID_ACTIVE`; altrimenti entra in `MID_WAITING`, se necessario, conservando l'apertura precedente durante T1. |
| $p = L_2$ | Mantiene il 100% se già in `HIGH`; altrimenti segue la gestione della fascia intermedia. |
| $p > L_2$ | Passa a `HIGH` e imposta immediatamente l'apertura al 100%. |

**Tabella 4:** Comportamento della politica automatica in funzione del livello e dello stato precedente.

In `MID_WAITING`, al raggiungimento di T1, la politica passa a `MID_ACTIVE` e imposta l'apertura al 50%. La scadenza viene verificata periodicamente dal CUS, anche in assenza di nuove misure.

Durante l'attesa viene mantenuta l'apertura precedente. Ad esempio, dopo una discesa da `HIGH` nella fascia intermedia, il 100% rimane attivo fino alla scadenza di T1, quando l'apertura passa al 50%.

Le principali transizioni della politica automatica sono rappresentate nella Figura 5.




**Condizioni e transizioni aggiuntive:**

- **Soglia L1:** con $p < L_1$ si passa a LOW. A $p = L_1$, MID_ACTIVE mantiene il 50%, mentre gli altri stati passano a LOW.
- **Soglia L2:** con $p > L_2$ si passa a HIGH, che rimane attivo anche a $p = L_2$.
- **Timer T1:** alla scadenza, MID_WAITING passa a MID_ACTIVE, anche senza una nuova misura.
- **Reset:** riporta la politica allo stato UNKNOWN.

**Figura 5:** Stati e principali transizioni della politica automatica.

### 4.2.2 Misure non valide, timeout e recupero

Quando il TMS segnala una lettura non valida tramite `NO_ECHO`, il CUS registra un campione `null` nello storico e rende il livello corrente non disponibile. In modalità `AUTOMATIC`, la politica viene azzerata, mantenendo invariata l'apertura della valvola.

Il timer T2 viene aggiornato soltanto alla ricezione di misure valide. Se non ne arrivano per almeno T2, il CUS passa allo stato `UNCONNECTED` e imposta l'apertura al 5%. Questo valore rappresenta un compromesso progettuale per consentire un deflusso limitato, evitando al tempo stesso uno svuotamento rapido della vasca.

Alla prima misura valida dopo la scadenza di T2, il CUS ripristina la modalità `AUTOMATIC` e rivaluta il livello. La rivalutazione parte internamente dall'apertura di fallback al 5%, senza necessariamente trasmettere al WCS un comando intermedio a tale valore.

### 4.2.3 Comandi manuali e storico

Il passaggio alla modalità `MANUAL` mantiene l'apertura corrente della valvola. L'operatore può impostare un'apertura compresa tra 0% e 100%, tramite l'interfaccia locale o la dashboard. Le misure del TMS continuano ad aggiornare il livello e lo storico, senza modificare l'apertura comandata.

Il ritorno alla modalità `AUTOMATIC` azzera la politica di controllo e rivaluta l'ultima misura, se disponibile. Durante lo stato `UNCONNECTED`, le richieste di cambio modalità e di regolazione dell'apertura vengono rifiutate.

Lo storico conserva in memoria gli ultimi $N$ campioni ricevuti e accettati, comprese le misure non valide rappresentate da `null`. I campioni sono ordinati cronologicamente in base alla ricezione da parte del CUS. Lo storico non è persistente e viene azzerato al riavvio.

## 4.3 Comunicazione e sincronizzazione

Il CUS gestisce eventi provenienti da MQTT, HTTP, comunicazione seriale e timer periodici, che possono essere elaborati concorrentemente. L'accesso allo stato condiviso, alla politica automatica e allo storico è protetto da un lock, mentre le operazioni di comunicazione vengono eseguite al di fuori delle sezioni critiche.

Lo stato del sistema è rappresentato attraverso snapshot immutabili, utilizzati per fornire informazioni coerenti ai componenti esterni.

Gli aggiornamenti destinati al WCS vengono accorpati per privilegiare lo stato più recente, evitando l'accumulo di stati intermedi. Le richieste ricevute tramite seriale vengono invece elaborate dal controller, che ne verifica la compatibilità con lo stato corrente e restituisce lo stato aggiornato.

## 4.4 Validazione, test e limitazioni

Il CUS è stato verificato attraverso test unitari JUnit, dedicati alla politica automatica, alla gestione dello stato e all'interpretazione dei messaggi MQTT.

I test coprono le transizioni tra modalità operative, il comportamento alle soglie L1 e L2, le scadenze T1 e T2, la gestione delle misure non valide e dei comandi manuali. Sono inoltre verificate la sincronizzazione degli aggiornamenti destinati al WCS e la coerenza del recupero dopo T2. Tutti i test unitari eseguiti sono stati superati.

Le verifiche unitarie non comprendono il funzionamento integrato con broker MQTT, dispositivi fisici e dashboard,che richiede prove separate.

Tra le limitazioni del prototipo, il CUS non dispone di un riscontro sulla posizione effettiva della valvola. La comunicazione seriale non prevede meccanismi automatici di riconnessione e ritrasmissione dei messaggi non consegnati. I servizi HTTP e MQTT non implementano autenticazione o cifratura delle comunicazioni.

# 5 Water Channel Subsystem (WCS)

Il WCS gestisce l'attuatore e l'interfaccia operatore locale. Riceve dal CUS lo stato del sistema e il comando di apertura della valvola, aggiornando di conseguenza il servomotore e il display LCD. Attraverso pulsante e potenziometro consente inoltre all'operatore di inviare richieste al CUS, senza modificare autonomamente lo stato globale.

## 5.1 Architettura e hardware

Il WCS è implementato su Arduino UNO con firmware C++ basato sul framework Arduino. La classe `HWPlatform` gestisce l'accesso ai dispositivi hardware, mentre `Context` mantiene l'ultimo stato ricevuto dal CUS, comprendente modalità operativa, apertura comandata e disponibilità dello stato.

I collegamenti dei componenti sono riportati nella Tabella 5 e rappresentati nello schema della Figura 6.

| Pin | Componente e funzione |
|---|---|
| D8 | Pulsante con circuito RC per attenuare i rimbalzi del segnale. |
| A0 | Ingresso analogico del potenziometro. |
| D9 | Segnale di controllo del servomotore. |
| A4 / A5 | Collegamenti SDA / SCL del display LCD I2C. |
| USB | Comunicazione seriale con il CUS a 115200 baud. |

**Tabella 5:** Collegamenti hardware del WCS.

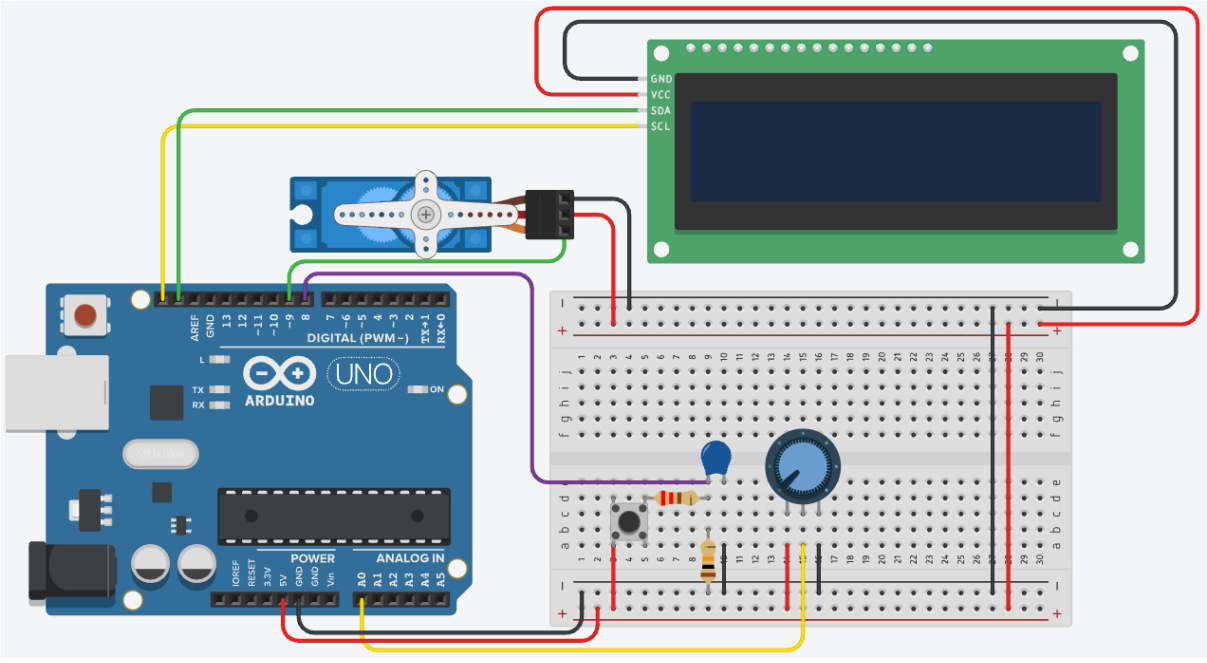



**Figura 6:** Schema dei collegamenti del prototipo WCS realizzato con Tinkercad.

## 5.2 Scheduler, task e macchine a stati

Il firmware è organizzato in task, ciascuno dedicato a una specifica funzione del WCS. La loro esecuzione è gestita da uno scheduler cooperativo, che li richiama in sequenza secondo i rispettivi periodi. Timer1 fornisce il riferimento temporale con un intervallo di base di 50 ms.

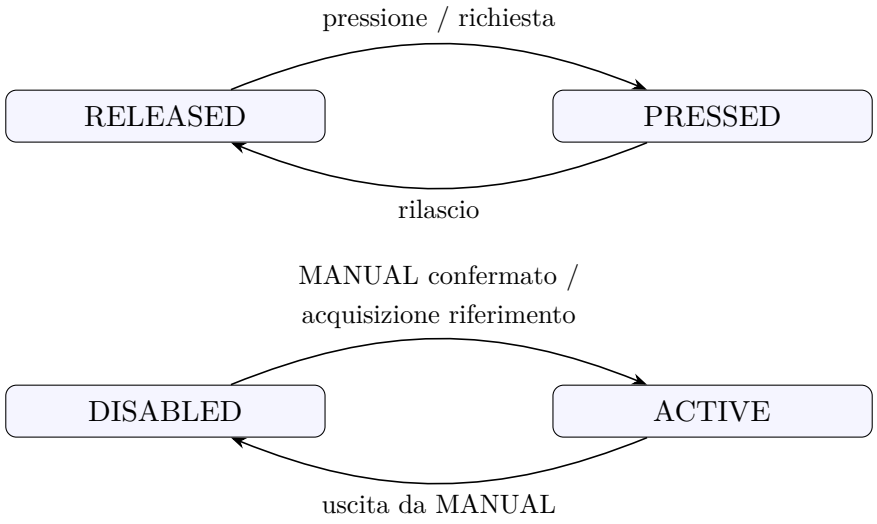
I task, le rispettive responsabilità e i periodi di esecuzione sono riportati nella Tabella 6.

| Task | Periodo | Responsabilità |
|---|---|---|
| SerialCommTask | 50 ms | Gestione della comunicazione seriale con il CUS. |
| ButtonTask | 50 ms | Rilevamento delle pressioni e richieste di cambio modalità. |
| PotentiometerTask | 100 ms | Lettura del potenziometro e richieste di apertura. |
| ValveTask | 50 ms | Aggiornamento del servomotore. |
| DisplayTask | 200 ms | Aggiornamento delle informazioni sul display LCD. |

**Tabella 6:** Task del WCS e relativi periodi di esecuzione.

I task condividono il `Context`, che contiene lo stato ricevuto dal CUS. Timer1 è dedicato allo scheduler, mentre Timer2 viene utilizzato dalla libreria `ServoTimer2` per il controllo del servomotore.

La gestione del pulsante e del potenziometro è organizzata attraverso due FSM, rappresentate nella Figura 7.




**Condizioni operative:**

- **Pulsante:** una richiesta per pressione, senza ripetizioni fino al rilascio.
- **Potenziometro:** in ACTIVE invia una richiesta per variazioni di almeno 5 punti percentuali o al raggiungimento di un estremo con variazione non nulla.
- **Stati iniziali:** RELEASED e DISABLED.

**Figura 7:** Macchine a stati per la gestione del pulsante e del potenziometro.

`ButtonTask` distingue gli stati `RELEASED` e `PRESSED`, generando una richiesta al rilevamento della pressione. Il circuito RC attenua i rimbalzi del pulsante, mentre la FSM evita richieste

ripetute finché il segnale rimane attivo. Prima della sincronizzazione con il CUS e durante `UNCONNECTED`, i comandi locali sono disabilitati.

`PotentiometerTask` opera invece negli stati `DISABLED` e `ACTIVE`. Quando il CUS conferma l'ingresso in `MANUAL`, il task acquisisce la posizione corrente del potenziometro come riferimento, senza inviare immediatamente un comando.

Le successive variazioni della manopola generano richieste di apertura quando raggiungono una soglia minima di spostamento. Sono inoltre gestiti gli estremi di apertura, per consentire il raggiungimento completo dello 0% e del 100%.

## 5.3 Interfaccia operatore e controllo della valvola

All'avvio il WCS mantiene il servomotore in posizione di chiusura e il display mostra un messaggio di attesa. Prima di rendere operativi i controlli locali, il sistema si sincronizza con il CUS e acquisisce uno stato valido.

Il pulsante permette di richiedere il cambio tra le modalità `AUTOMATIC` e `MANUAL`. In modalità `MANUAL`, il potenziometro consente di richiedere una diversa apertura della valvola.

Le richieste vengono inviate al CUS, che ne determina l'accettazione. Il WCS aggiorna servomotore e display soltanto in base allo stato successivamente ricevuto, mantenendo il controllo centralizzato del sistema.

Il task `ValveTask` converte la percentuale di apertura comandata, compresa tra 0% e 100%, in un angolo di comando del servomotore tra 0° e 90°.

Il display LCD mostra la modalità operativa e la percentuale di apertura della valvola. Le informazioni vengono aggiornate quando cambia lo stato ricevuto.

## 5.4 Validazione, test e limitazioni

Il WCS è stato sottoposto a prove funzionali sul prototipo fisico, verificando l'avvio e la sincronizzazione con il CUS, la gestione del pulsante, la regolazione tramite potenziometro, il movimento del servomotore e l'aggiornamento del display LCD.

Le prove hanno confermato il comportamento previsto dei componenti e dell'interfaccia locale, compreso l'ingresso in modalità `MANUAL` senza modificare immediatamente l'apertura della valvola. È stato inoltre verificato il funzionamento della comunicazione seriale con il CUS.

Tra le limitazioni, il WCS non dispone di un feedback sulla posizione effettiva del servomotore. Non è inoltre prevista una gestione automatica delle interruzioni della comunicazione seriale: il WCS può mantenere l'ultimo stato ricevuto e il ripristino del collegamento può richiedere il riavvio del CUS e del WCS.

# 6 Dashboard Subsystem (DBS)

Il DBS permette di monitorare e controllare il sistema attraverso un'interfaccia web. Visualizza la modalità operativa, il livello dell'acqua, l'apertura comandata della valvola e lo storico delle misure. Consente inoltre all'operatore di inviare richieste al CUS, che mantiene il controllo dello stato globale.

## 6.1 Architettura dell'interfaccia web

Il DBS è sviluppato con HTML, CSS e JavaScript, senza l'utilizzo di framework frontend. L'interfaccia comprende pannelli informativi, controlli per l'operatore e un grafico per la visualizzazione dello storico.

Il codice JavaScript è organizzato in moduli che separano la comunicazione HTTP, la gestione dell'interfaccia e l'aggiornamento del grafico. Quest'ultimo utilizza Chart.js, incluso localmente nel progetto.

## 6.2 Comunicazione HTTP e gestione dello stato

Il DBS comunica con il CUS tramite HTTP, recuperando periodicamente lo stato del sistema e lo storico delle misure. Nella configurazione attuale, le richieste sono indirizzate a `http://localhost:8080`, con intervalli di aggiornamento nominali di 1000 ms per entrambi i dati.

L'interfaccia distingue le misure non disponibili, rappresentate da `null`, da un livello valido pari allo 0%. Nel grafico, le misure non valide sono rappresentate mediante interruzioni della linea.

La dashboard distingue inoltre due condizioni di indisponibilità. Lo stato `UNCONNECTED` viene comunicato dal CUS quando non riceve misure valide dal TMS per un intervallo prolungato. La condizione `NOT AVAILABLE` indica invece che il DBS non riesce a ottenere uno stato utilizzabile dal CUS.

In quest'ultimo caso, i comandi vengono disabilitati e gli ultimi valori disponibili rimangono visualizzati, senza essere considerati informazioni aggiornate. Un errore nel recupero del solo storico viene invece segnalato separatamente, senza modificare lo stato di disponibilità del CUS.

## 6.3 Interazione dell'operatore

L'interfaccia mette a disposizione un pulsante per richiedere il cambio tra le modalità `AUTOMATIC` e `MANUAL` e uno slider per regolare l'apertura della valvola.

Lo slider è abilitato soltanto in modalità `MANUAL`, quando il CUS è disponibile. Durante la regolazione viene mostrato il valore selezionato, mentre la richiesta di apertura viene inviata al termine della modifica.

Le richieste dell'operatore vengono elaborate dal CUS. La dashboard aggiorna i valori del sistema sulla base dello stato restituito dal backend, senza considerare automaticamente applicato un comando appena inviato.

Durante l'elaborazione delle richieste, i relativi controlli vengono temporaneamente disabilitati.

## 6.4 Validazione, test e limitazioni

Il DBS è stato sottoposto a prove funzionali, verificando la visualizzazione dello stato del sistema, del livello dell'acqua, dell'apertura comandata e dello storico delle misure.

Le prove hanno confermato il funzionamento del cambio di modalità, della regolazione tramite slider e dell'aggiornamento del grafico. È stata inoltre verificata la gestione dell'indisponibilità del CUS e del successivo ripristino della comunicazione.

Tra le limitazioni, la dashboard aggiorna le informazioni periodicamente e non rappresenta necessariamente lo stato istantaneo del sistema. Inoltre, non verifica direttamente la disponibilità del WCS né la posizione effettiva della valvola.

# 7 Integrazione e comunicazione

## 7.1 Interazione tra sottosistemi e contratti

I sottosistemi comunicano attraverso tre protocolli: MQTT tra TMS e CUS, seriale tra WCS e CUS e HTTP tra DBS e CUS.

Nel funzionamento automatico, il TMS invia le misure attraverso il broker MQTT. Il CUS le elabora, aggiorna lo stato del sistema e, quando necessario, trasmette al WCS il nuovo comando di apertura. Il WCS aggiorna di conseguenza il servomotore e il display LCD.

Il DBS consulta periodicamente lo stato e lo storico tramite HTTP, indipendentemente dall'aggiornamento del WCS. La Figura 8 rappresenta il flusso delle informazioni tra i sottosistemi.

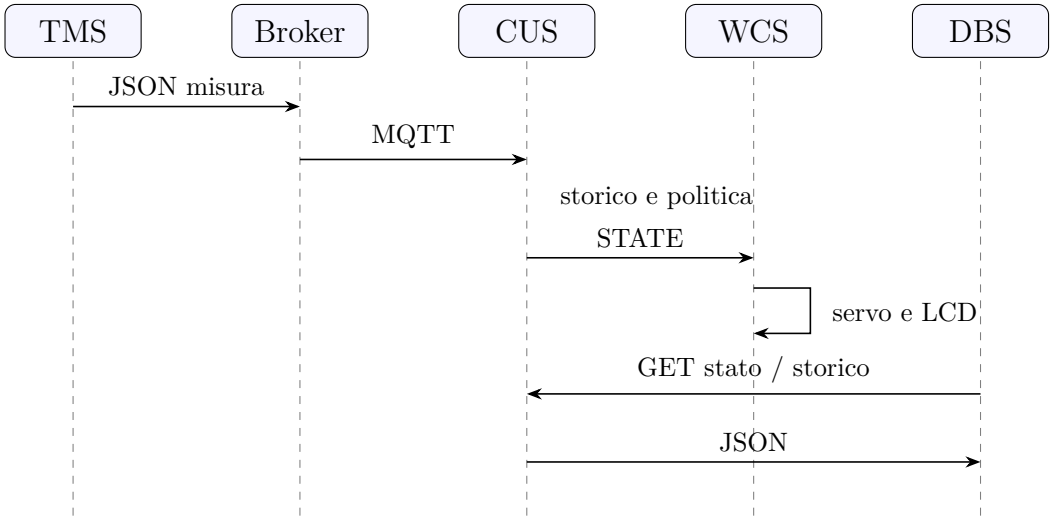



**Figura 8:** Sequenza logica di una misura che determina un aggiornamento del sistema. La consultazione del DBS è indipendente dall'attuazione sul WCS

**MQTT.** Il TMS pubblica le misure su un topic MQTT condiviso con il CUS, utilizzando messaggi JSON. Una misura valida contiene lo stato `OK`, il livello dell'acqua in metri e la percentuale di riempimento. Una lettura non valida viene invece segnalata attraverso `NO_ECHO`.

Il CUS utilizza la percentuale ricevuta per la politica automatica e assegna il timestamp ai campioni quando vengono ricevuti. La comunicazione utilizza QoS 0 e non prevede una conferma applicativa dell'elaborazione della misura.

**Seriale.** La comunicazione tra WCS e CUS utilizza un protocollo testuale a 115200 baud. Il WCS invia richieste di sincronizzazione e comandi dell'operatore, mentre il CUS trasmette gli aggiornamenti dello stato.

I principali messaggi sono riportati nella Tabella 7.

I comandi inviati dal WCS vengono valutati dal CUS, che restituisce lo stato corrente del sistema. Il messaggio `STATE` rappresenta quindi lo stato definito dal controller, non una conferma dell'effettiva posizione del servomotore.

| Messaggio | Significato |
|---|---|
| `READY` | Richiesta di sincronizzazione dal WCS |
| `CMD,MODE,...` | Richiesta di cambio modalità dal WCS |
| `CMD,VALVE,...` | Richiesta di apertura dal WCS |
| `STATE,...` | Stato corrente trasmesso dal CUS al WCS |
| `LOG,...` | Messaggio diagnostico opzionale del WCS |

**Tabella 7:** Principali messaggi del protocollo seriale tra WCS e CUS

**HTTP.** Il DBS utilizza le API HTTP del CUS per consultare lo stato del sistema, recuperare lo storico delle misure e inviare richieste dell'operatore.

Le operazioni disponibili sono riportate nella Tabella 8.

| Operazione | Funzione |
|---|---|
| `GET /api/state` | Restituisce modalità, apertura e livello corrente. |
| `GET /api/measurements` | Restituisce gli ultimi $N$ campioni dello storico. |
| `POST /api/mode` | Richiede il cambio di modalità operativa. |
| `POST /api/valve` | Richiede una nuova apertura in modalità MANUAL. |

**Tabella 8:** Operazioni HTTP esposte dal CUS e utilizzate dal DBS

Le richieste di modifica vengono accettate o rifiutate dal CUS in base allo stato corrente. Il DBS aggiorna la propria rappresentazione utilizzando le informazioni restituite dal backend.

## 7.2 Scenari operativi e gestione delle interruzioni

**Avvio e sincronizzazione.** All'avvio il CUS inizializza lo stato in modalità `AUTOMATIC`, mentre il WCS mantiene la valvola chiusa fino alla ricezione di uno stato valido. La sincronizzazione avviene attraverso lo scambio dei messaggi `READY` e `STATE`.

Il DBS acquisisce autonomamente lo stato tramite HTTP, abilitando i controlli in base alla modalità e alla disponibilità del CUS.

**Controllo automatico.** Le misure del TMS vengono elaborate dal CUS secondo la politica descritta nella Sezione 4.2. Le variazioni dell'apertura vengono comunicate al WCS, mentre il DBS rileva gli aggiornamenti attraverso le proprie richieste periodiche.

La scadenza di T1 può determinare una variazione dell'apertura anche senza la ricezione di una nuova misura.

**Controllo locale e remoto.** L'operatore può richiedere il cambio di modalità e la regolazione dell'apertura attraverso il WCS o il DBS. Entrambe le interfacce inviano le richieste al CUS, che gestisce lo stato condiviso e determina il comando effettivo.

Non è prevista una priorità fissa tra operatore locale e remoto. Le richieste concorrenti vengono gestite dal controller e le interfacce si aggiornano in base allo stato ricevuto.

**Indisponibilità delle misure.** In assenza di misure valide per almeno T2, il CUS passa allo stato `UNCONNECTED` e imposta l'apertura al 5%. La condizione viene comunicata al WCS e resa disponibile al DBS.

I messaggi `NO_ECHO` indicano invece una misura non valida: rendono il livello corrente non disponibile e azzerano la politica automatica, senza modificare immediatamente l'apertura né rinnovare T2.

Alla prima misura valida dopo T2, il CUS ripristina `AUTOMATIC` e rivaluta il livello a partire dal fallback interno al 5%.

**Interruzioni delle comunicazioni.** Se il DBS non riesce a comunicare con il CUS, mostra la condizione `NOT AVAILABLE`, senza modificare lo stato degli altri sottosistemi.

Un'interruzione della comunicazione seriale non impedisce al CUS di continuare a elaborare le misure del TMS e le richieste HTTP, ma il WCS può mantenere l'ultimo comando ricevuto.

Analogamente, l'indisponibilità del TMS non implica necessariamente quella del CUS o della dashboard. Le diverse condizioni vengono quindi gestite separatamente.

## 7.3 Validazione end-to-end e copertura dei requisiti

La validazione del progetto comprende test unitari del CUS e prove funzionali dei sottosistemi hardware e dell'interfaccia web, descritti nei rispettivi capitoli.

Sono stati verificati il funzionamento dei controlli locali del WCS, la comunicazione con il CUS, l'interazione tramite dashboard e la gestione delle condizioni di indisponibilità e ripristino. Le verifiche del CUS comprendono inoltre la politica automatica, i timeout e la gestione delle richieste.

Queste prove forniscono riscontri sul funzionamento dei singoli sottosistemi e di alcune loro interazioni. Non è tuttavia disponibile una documentazione sistematica di una verifica end-to-end completa, dall'acquisizione della misura sul TMS fino all'attuazione fisica e alla visualizzazione simultanea sul DBS.

La validazione complessiva rimane quindi limitata alle funzionalità e alle condizioni effettivamente verificate, senza estendere i risultati alle situazioni non sottoposte a prova.

# 8 Conclusioni

## 8.1 Risultati e limitazioni

Il progetto ha portato alla realizzazione di un sistema distribuito per il monitoraggio del livello dell'acqua e il controllo dell'apertura di una valvola. L'architettura adottata affida al CUS la gestione centralizzata dello stato e della politica di controllo, integrando acquisizione delle misure, attuazione e interfacce operatore locali e remote.

La validazione ha compreso test unitari del CUS, superati con esito positivo, e prove funzionali del WCS e del DBS. Sono state inoltre effettuate verifiche qualitative sul TMS. Questi risultati confermano i comportamenti verificati, ma non costituiscono una validazione sistematica dell'intero sistema in tutte le condizioni operative.

Le principali limitazioni riguardano l'assenza di un feedback sulla posizione effettiva della valvola e il rilevamento delle interruzioni della comunicazione seriale. La disponibilità dei singoli sottosistemi non garantisce inoltre, da sola, il corretto funzionamento dell'intera catena di controllo.

## 8.2 Possibili sviluppi futuri

Tra i possibili miglioramenti rientrano l'introduzione di un feedback sulla posizione della valvola, il monitoraggio della comunicazione tra CUS e WCS, la persistenza dello storico delle misure e l'autenticazione delle richieste di controllo.

Queste estensioni potrebbero migliorare l'affidabilità e la gestione del sistema, pur non rientrando negli obiettivi dell'attuale implementazione.

# Riferimenti bibliografici

[1] A. Ricci, *Assignment #03 – Smart Tank Monitoring System*, Embedded Systems and Internet of Things, a.a. 2025/2026, versione 1.0.0-20251112. Consegna ufficiale nel repository del corso. Consultato il 3 novembre 2026.